MGAI Assignment 1: Procedural Content Generation Report

Yutao Liu
Leiden University
s4213211
y.liu.67@umail.leidenuniv.nl

Abstract

This report presents an adaptive house generation system that creates stylistically appropriate buildings based on terrain type, incorporates randomized elements for variation, and ensures structures integrate naturally with the environment.

Introduction

Procedural Content Generation (PCG) is an important research direction in game development and computer graphics. It automatically generates content through algorithms to improve the playability and diversity of games. In open world games, PCG technology can be used to automatically generate terrain, buildings, tasks, characters and other content, making each game experience unique and varied. Minecraft, as a highly free sandbox game, provides an ideal experimental environment for PCG research.

In this assignment, we will use the GDPC (Generative Design in Minecraft Python Client) framework (GDPC Documentation Team 2025) to automatically generate houses with a specific architectural style in Minecraft and ensure that they can adapt to different terrain environments.

Related Work

PCG techniques have been widely used in video games to create dynamic environments, levels, and architectures. (Patterson and Ward 2022) provide an overview of PCG approaches, categorizing them into constructive, search-based, and machine learning-driven approaches; a key challenge in PCG is to ensure that procedurally generated structures blend naturally into existing terrain. (Salge et al. 2018) launched the GDMC (Generative Design in Minecraft) competition, which evaluates AI-driven settlements based on adaptability, functionality, and aesthetics; Minecraft presents unique challenges for PCG due to its voxel-based, open-world nature. (Latif et al. 2022) proposed an approach in which AI-driven builders follow natural language instructions to construct procedurally generated structures. Other studies, such as (Kiseleva et al. 2021), have integrated reinforcement learning (RL) to enable AI agents to adapt to dynamic environments.

Methodology

Terrain Analysis and Adaptive Building

To ensure that buildings are placed in the best position, we use a multi-step terrain evaluation method based on heightmaps, slope analysis, and natural element detection. We use `editor.worldSlice.heightmaps` to get the terrain heightmap, and use `numpy.gradient()` to calculate the height change on the X and Z axes to evaluate the slope of the terrain.

Areas with too steep a slope will be marked as unsuitable for construction. We use the **Matplotlib** to visualize the terrain suitability score. Before the final site selection, the script checks whether `editor.getBlock()` returns "water" to prevent the building from floating on water. If the ground is uneven, the foundation is automatically adjusted to fill the height difference with **stone** or **oak_log**.

Once the appropriate area is determined, the houses are placed programmatically without overlapping. First, the program randomly selects multiple locations within the buildable area; it uses the **s4213211_isOverlap()** function to check if there is overlap with an existing building, and if so, it tries to find a new location up to 30 times. Then, the program uses **s4213211_adaptiveFoundation()** to calculate the average height of the terrain and adjust the foundation. If the terrain is low, fill it with stones to raise the house. Otherwise, clear the high part to level the ground. Finally, the **s4213211_selectBuildingStyle()** function helps the program automatically select a building style, such as "forest style" in forest terrain. The **s4213211_visualizeBuildingLocation()** function is used to verify the site selection to ensure that the houses adapt to the terrain and are reasonably distributed.

Figure 1 is a visualization of the terrain analysis of the current world map and a terrain suitability analysis map obtained based on calculations: red areas indicate steep slopes and are not suitable for construction; yellow areas indicate areas that can be built, but the terrain may be slightly undulating; green areas indicate flat terrain, which is most suitable for construction.The red boxes represent houses that have been built.

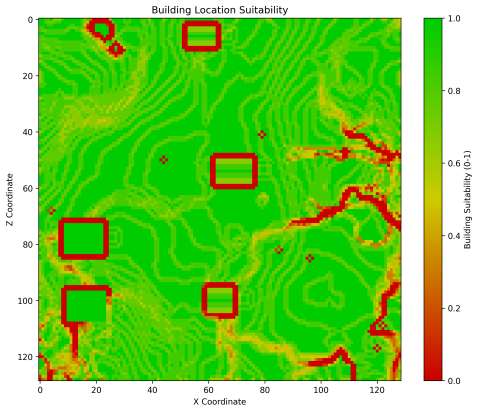

Figure 1: Building Suitability Score

Building Style Selection

In order to make the procedurally generated buildings blend in with the environment, we need to match the randomly generated terrain style in Minecraft. Finally, we determined three different building styles that can be achieved: forest style for grassland and forest areas, desert style for desert and arid areas, and stone style for mountain and rock areas. The selection process is automated. We use the function **s4213211_selectBuildingStyle()** to determine the building style based on the terrain material:

Algorithm 1 `s4213211_selectBuildingStyle`

procedure SELECTBUILDINGSTYLE(editor, x, z, heightmap, $x_{\min}$, $z_{\min}$)
  $local\_x \leftarrow x - x_{\min}$
  $local\_z \leftarrow z - z_{\min}$
  $y \leftarrow$ **int**(heightmap$[local_x, local_z]) - 1$
  $ground\_block \leftarrow$ editor.getBlock(x, y, z)
  if "sand" in ground_block **or** "sandstone" in ground_block **then**
    return "desert"
  else if "stone" in ground_block **or** "andesite" in ground_block **or** "granite" in ground_block **then**
    return "stone"
  else
    return "forest"
  end if
end procedure

Once the building style is determined, the **s4213211_get BuildingMaterials()** function automatically assigns appropriate materials, including walls, roofs, windows, floors, porches, and decorative elements(Fig.2 as an example).

Figure 2: Different Building Style

Randomness and Diversity

To ensure that procedurally generated buildings can reflect randomness and diversity, the program introduces `randint()` in many steps when building houses. First, the width, depth, and height of each house are randomly determined. The possible house sizes range from 6x8 to 10x12.Some houses may be single-floor buildings, while others are high-rise buildings(Fig.2 as an example).

Algorithm 2 `s4213211_generateHouse()`

function GENERATEBUILDINGSIZE
  $width \leftarrow Random(6, 10)$
  $depth \leftarrow Random(8, 12)$
  $floor\_height \leftarrow 3$
  $floor\_count \leftarrow Random(2, 4)$
  $height \leftarrow floor\_count \times floor\_height$
  return $(width, depth, height)$
end function

Figure 3: Different house sizes

For the choice of roof, flat roofs or pitched roofs are generated with a 50%/50% probability; the **s4213211_get BuildingMaterials()** function ensures that each house uses different wall, floor and decoration materials and windows, and matches the previously determined architectural style; the position of the door is relatively fixed, but it ensures that the overall style of the house is diverse. Here are some examples:

Figure 4: Different Roofs

Figure 4 shows the different roofs were generated during the built;

Figure 5: Interior Example(a)

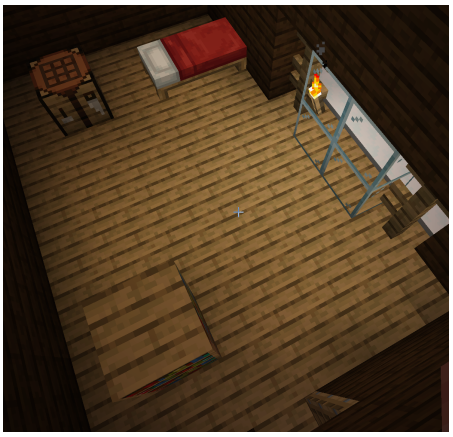

Figure 6: Interior Example(b)

Algorithm 3 `s4213211_generateStyledWindows()`

function GENERATESTYLESWINDOWS(editor, x, y, z, $width$, $height$, $depth$, $building\_style$)
  $win\_width \leftarrow 2$
  $win\_height \leftarrow 2$
  $win\_x \leftarrow Random(x{+}1, x{+}width{-}win\_width{-}1)$
  $win\_y \leftarrow Random(y + 1, y + height - win\_height - 1)$
  if $building\_style = "desert"$ **then**
    Place Colored Stained Glass
  else if $building\_style = "stone"$ **then**
    Randomly Choose Between Iron Bars or Glass
  else
    Place Standard Glass
  end if
end function

Figure 5&6 shows the different interior decoration were generated during the built, based on their building styles;Algorithm 3 briefly explains the logic of window generation.

Environmental Integration

Making the generated buildings look natural and well integrated with the environment is an extremely critical step, which proves that our procedurally generated buildings are based on real-world logic, rather than built out of thin air.

Figure 7: Generated Wood House blend in the forest

The **s4213211_adaptiveFoundation()** function ensures that the houses are not suspended in the air and automatically adjusts the foundations to prevent them from floating on the ground. At the same time, it automatically fills in low-lying areas and appropriately flattens high-lying areas. **s4213211_generateGarden()** generates a 4*4 fenced garden on one side of the house, providing a buffer zone so that the house naturally transitions to the surrounding environment.

Figure 8: Generated Garden

If the house is close to water, the program will automatically detect the water level and use wooden pillars to support it to ensure that the house is not built directly on the water surface, improving authenticity.

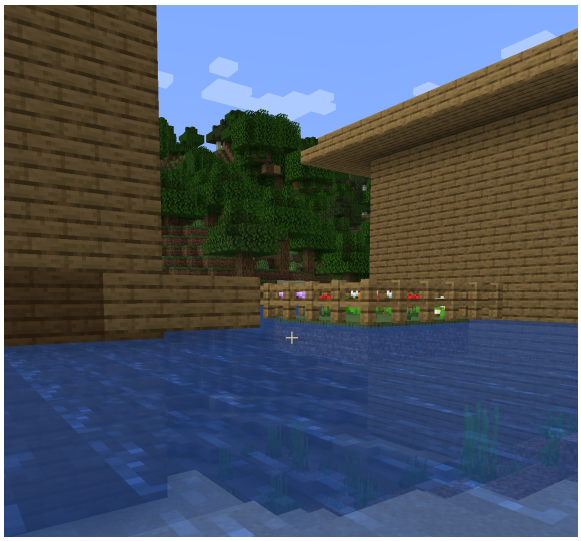

Figure 9: Generated House in water surface

Results

After the program is running, in addition to the previous Suitability Score(Fig.1), the corresponding 3D terrain map(Fig.10) and building location map(Fig.11) will also be generated. The corresponding houses will also be generated at the same time.

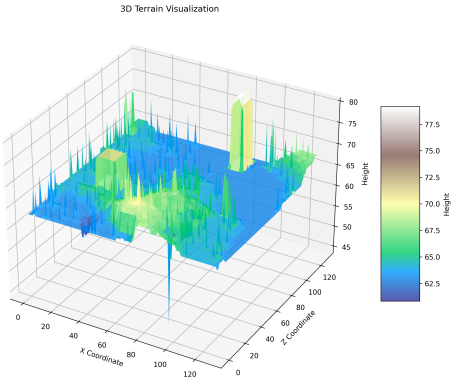

Figure 10: 3D Terrain

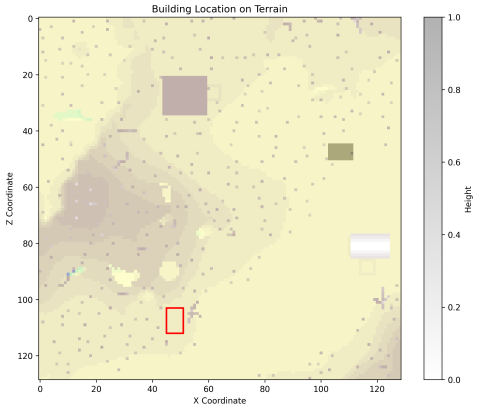

Figure 11: Building Location

Figure 12: Generated House

Overall, It effectively ensures the adaptability of buildings to terrain, structural diversity and environmental integration:

- **Adaptive Foundation System:** Adjusts the building height to ensure that the building does not float or sink into the ground. The foundation is automatically filled to create stable support on slopes or water surfaces.

- **Size and Material Diversity:** The size variation makes each house different in size, and the randomized materials make the buildings look different in style.The garden connects the building to the natural environment.

- **Biome-Based Material Matching:** Ensures that the building matches the environment by selecting materials based on the biome.

- **Overlap Detection:** Ensures that the houses do not overlap each other and maintains reasonable spacing.

- **3D Terrain and Building Visualization:** Assists in analyzing the suitability of the terrain.

Although the system can efficiently create houses of different styles, there are still some limitations. For example, the house design is relatively simple, lacking multi-room layouts or complex architectural appearances; the interior decoration only provides basic furniture and a small number. In terms of biome adaptation, since the material selection is preset, the program cannot dynamically analyze the density of surrounding blocks. Furthermore, while randomness increases diversity, houses in the same biome may still be similar.

Finally, the program lacks global village layout logic, and the structures between houses are independent of each other and lack interaction, making it impossible to generate an overall planned village.

Discussion

Balancing Randomness and Structured Design

In PCG, the balance between randomness and structural rules is an important challenge. Complete randomization can lead to chaotic, unrealistic designs, while overly restrictive rules reduce diversity. Therefore, the randomization implemented in this assignment (such as house size, building materials, etc.) has a certain range.

PCG buildings match real-world logic

To make procedurally generated buildings believable, it is important to match real-world logic. Make sure that houses do not float, are buried in the ground, or are suspended in the air, and that the appearance of the building matches the biome. PCG that does not conform to real-world logic will appear unnatural and unreal. Here are some examples of illogical errors encountered when generating buildings:

Figure 13: Illogical Errors(a-b)

Figure 14: Illogical Errors(c)

Directions for future works

For building styles, materials for snow, swamp, and ocean biomes should be expanded to accommodate more biome styles;

For buildings themselves, add room dividers, furniture randomization, and interior details;

For building communities, settlements should be organized more realistically, and Village-Level cohesion should be enhanced;

Use spatial partitioning algorithms to improve overlap detection efficiency, optimize performance, and improve the problem that the current collision detection function has a large amount of calculations in large villages, which may reduce the generation speed.

References

[GDPC Documentation Team 2025] GDPC Documentation Team. 2025. Building a house - gdpc tutorial. Accessed: 2025-02-26.

[Kiseleva et al. 2021] Kiseleva, J.; Li, Z.; Aliannejadi, M.; Mohanty, S.; et al. 2021. Neurips 2021 competition iglu: Interactive grounded language understanding in a collaborative environment. *arXiv preprint*.

[Latif et al. 2022] Latif, A.; Zuhairi, M. F.; Khan, F. Q.; Randhawa, P.; and Patel, A. 2022. A critical evaluation of procedural content generation approaches for digital twins. *Wiley Online Library*.

[Patterson and Ward 2022] Patterson, B., and Ward, M. 2022. Adaptive procedural generation in minecraft. *Temple University, CIS 5603 Final Project*.

[Salge et al. 2018] Salge, C.; Green, M. C.; Canaan, R.; and Togelius, J. 2018. Generative design in minecraft (gdmc) settlement generation competition. *Proceedings of the 13th International Conference on the Foundations of Digital Games* 1–10.